EXPERIMENT 9

NAME:DEVISHREE J **ROLLNO:240701702**

Title

Design input forms that validate data (email, phone number) and display error messages using HTML, CSS, and JavaScript with Validator.js.

AIM

To design an input form that validates user data such as email and phone number and displays appropriate error messages using HTML, CSS, and JavaScript with Validator.js.

TOOLS REQUIRED

- HTML
- CSS
- JavaScript
- Validator.js Library
- VS Code / Any Code Editor
- Web Browser (Chrome)

PROCEDURE

Step 1: Create HTML Form

- Create a form with input fields for email and phone number
- Add `<span>` elements to display error messages
- Link CSS and JavaScript files

Step 2: Apply CSS Styling

- Style the form using CSS
- Add spacing, colors, and layout
- Highlight error messages in red

Step 3: Add JavaScript Validation

- Use Validator.js library
- Validate email using `isEmail()`
- Validate phone number using `isMobilePhone()`
- Display error messages dynamically
- Prevent form submission if invalid

PROGRAM

HTML Code

```
<!DOCTYPE html>
<html>
<head>
    <title>Form Validation</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
    <h2>Registration Form</h2>

    <form id="myForm">
        <label>Email:</label>
        <input type="text" id="email"
placeholder="Enter email">
        <span id="emailError"
```

```
class="error"></span>

        <label>Phone Number:</label>
        <input type="text" id="phone"
placeholder="Enter phone number">
        <span id="phoneError"
class="error"></span>

        <button type="submit">Submit</button>
    </form>
</div>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/val
idator/13.7.0/validator.min.js"></script>
<script src="script.js"></script>

</body>
</html>
```

CSS Code

```
body {
    font-family: Arial;
    background-color: #f4f4f4;
}

.container {
    width: 300px;
    margin: 100px auto;
    padding: 20px;
    background: white;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
}

input {
```

```
    width: 100%;
    padding: 8px;
    margin: 10px 0;
}

button {
    width: 100%;
    padding: 10px;
    background: blue;
    color: white;
    border: none;
}

.error {
    color: red;
    font-size: 12px;
    display: block;
    margin-bottom: 5px;
}
```

JavaScript Code

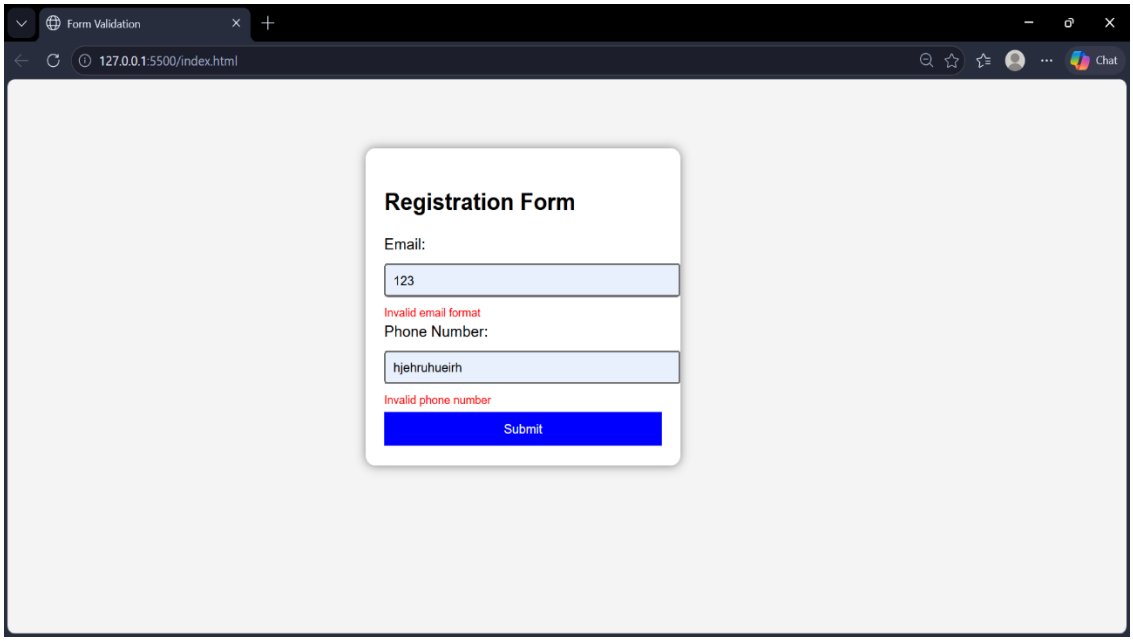
```
document.addEventListener("DOMContentLoaded",
function () {


document.getElementById("myForm").addEventListe
ner("submit", function(e) {

        e.preventDefault();

        let email =
document.getElementById("email").value;
        let phone =
document.getElementById("phone").value;

        let emailError =
```

```
document.getElementById("emailError");
        let phoneError =
document.getElementById("phoneError");

        emailError.textContent = "";
        phoneError.textContent = "";

        let valid = true;

        if (!validator.isEmail(email)) {
            emailError.textContent = "Invalid
email format";
            valid = false;
        }

        if (!validator.isMobilePhone(phone,
'en-IN')) {
            phoneError.textContent = "Invalid
phone number";
            valid = false;
        }

        if (valid) {
            alert("Form submitted
successfully!");
        }

    });

});
```

OUTPUT

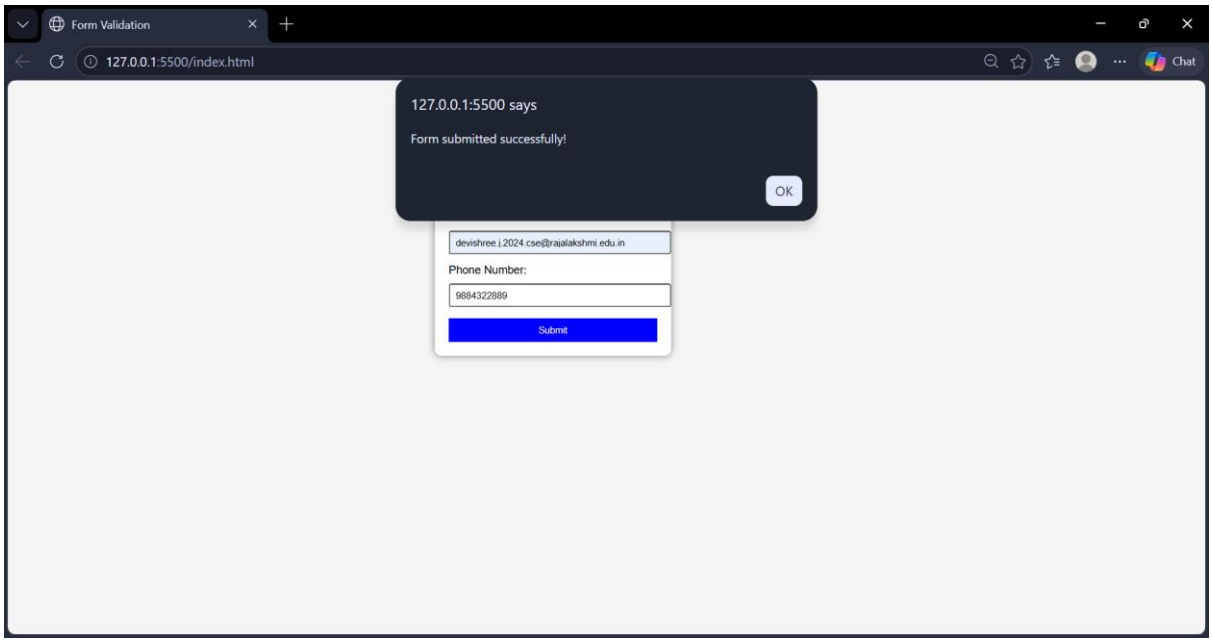
Case 1: Invalid Input

- When incorrect email or phone number is entered
- Error messages are displayed below input fields



Case 2: Valid Input

- When correct details are entered
- Form displays success alert message



RESULT

The input form was successfully designed and validated using HTML, CSS, and JavaScript with Validator.js. Error messages were displayed for invalid inputs, ensuring correct data entry.

CONCLUSION

This experiment demonstrates how client-side validation improves user experience and data accuracy by preventing incorrect inputs before form submission.